Linux Process Watchdog (Nexus) – Project Report

Oct 17, 2026 · @ak

Team: Bhanu Jakka (2520030006), Akshita Anand (2520030043), A. Kuvira (2520030192), P. Sudha (2520030455)

Abstract

Nexus is a Linux process watchdog that detects unexpected process termination, restarts the process under a bounded policy, verifies the recovery and records the incident. It is built with Python, FastAPI, psutil and SQLite on the backend and a React dashboard on the front. In local testing it detected a killed worker on the next poll, restarted it, passed verification and raised an alert, while refusing to run any script other than its allowlisted test worker.

1. Introduction

Long-running services can terminate without warning. Common causes are out-of-memory kills (SIGKILL), segmentation faults (SIGSEGV), unhandled exceptions, exhausted file descriptors or disk, and accidental kills. Without a watchdog the service stays down until an administrator notices, restarts it by hand and investigates the cause. This costs downtime and manual effort, and usually leaves no record of what happened.

1.1 Problem statement

To design a lightweight system that monitors selected Linux processes, restarts them automatically when they terminate unexpectedly, and keeps a verifiable record of every failure and recovery, without allowing the tool itself to become a way to run arbitrary commands.

1.2 Objectives

1. Continuously monitor registered processes with a background agent (default poll every 5 s).
2. Detect termination, including missing, zombie and wrong-identity processes.

3. Restart the allowlisted process automatically with no manual action.
4. Bound restarts with a per-application attempt limit and a cooldown.
5. Log every incident, attempt and alert in SQLite.
6. Present the data in a simple visual dashboard backed by FastAPI.

1.3 Scope

The current system targets a single Linux host and recovers only the disposable `test_process.py` worker. Real services, multiple hosts and authentication are outside the present scope (see Section 5).

2. Background and existing systems

2.1 Process fundamentals

A Linux process is in one of several states: running or runnable (R), interruptible sleep (S), uninterruptible I/O wait (D), stopped or traced (T) or zombie (Z, defunct but not yet reaped). Termination is usually the result of a signal: SIGTERM (15) is a polite stop request, SIGKILL (9) cannot be caught, SIGSEGV (11) signals invalid memory access, SIGINT (2) is an interrupt and SIGCHLD tells a parent that a child changed state. A PID alone is not a safe identifier because the kernel can reuse it, so Nexus also compares the process start time and command line.

2.2 Existing technologies

Technology	How it works	Strength	Typical gap
systemd `Restart=`	Restarts failed units by policy	Production grade	No incident view
supervisord	Auto-restarts managed processes	Simple control	Little incident history
Monit	Watches, restarts, sends alerts	Mature, lightweight	File-based, light history
Docker / Kubernetes	Restart policy and probes	Standard for containers	Needs a container platform

Technology	How it works	Strength	Typical gap
Prometheus / Zabbix	Metrics, dashboards, alerting	Rich observability	Alerts; recovery is extra

2.3 Proposed system

The restart mechanism itself is not new. What Nexus adds is a single console that restarts a process, verifies the restart and records each recovery as an incident with a timeline, restart attempts and alerts, while restricting what can be executed.

3. Design and implementation

3.1 Architecture

A React dashboard (Vite, TypeScript) reads data from a FastAPI backend over REST (`/api`). Inside the backend an asynchronous monitor loop runs every interval and uses a process checker (psutil), a restart manager and a policy engine. Incidents, restart attempts, alerts, applications, policies and monitoring history are stored in SQLite through SQLAlchemy. Settings (interval, database, CORS origins) come from environment variables.

3.2 Technology stack

Layer	Technology	Role
Operating system	Linux (Ubuntu / Debian)	Target kernel and process model
Backend	Python 3, FastAPI, Uvicorn	API and monitor loop
Process inspection	psutil	Identity, state, CPU, memory
Storage	SQLite, SQLAlchemy	Incidents, attempts, alerts, history
Frontend	React, TypeScript, Vite, Recharts	Dashboard over REST
Testing	pytest, curl	Backend tests and live checks

3.3 Detection method

Each cycle the checker marks a process alive only if all of the following hold: the PID exists, it is not a zombie, its creation time is within 2 s of the recorded value, and its command line contains `test_process.py`. An optional HTTP health check must return a status from 200 to 399 within 2 s. A process that is not alive while `expected_running` is true is marked Failed and an incident is opened; if `expected_running` is false it is marked Stopped and no incident is raised.

3.4 Recovery workflow

1. Start the agent and launch the monitor loop.
2. Poll every registered process (default 5 s, minimum 2 s).
3. Detect a missing, zombie or wrong-identity process.
4. Record: open an incident with a "Failure detected" event.
5. Restart: check that automatic restart is enabled and the attempt limit is not reached, respect the cooldown, then spawn the allowlisted script.
6. Verify and alert: the process must be alive within 3 s and pass the health check; the incident becomes Recovered and an alert is raised. If verification fails with no attempts left, the incident is Unresolved with a Critical alert.

3.5 Recovery policy

Control	Default	Effect
Allowlist	`test_process.py` only	Anything else is rejected at registration (HTTP 422)
Maximum attempts	3 (range 1 to 10)	Incident becomes Unresolved when exhausted
Cooldown	5 s in the demo setup	Minimum time between attempts
Verification	Alive within 3 s plus health check	A restart counts only when verified

3.6 Incidents, alerts and data model

Every incident stores its type, severity, status, detection and recovery times, recovery duration, number of attempts and a JSON timeline. Each restart is saved as an attempt with its outcome. An alert is raised for each recovery outcome with a severity from Low to Critical and can be acknowledged in the dashboard. Each poll also writes a monitoring-history row (status, CPU %, memory MB) used for analytics.

3.7 Self-protection

Only `test_process.py` can be executed, the frontend never supplies a shell command, process arguments are limited to short strings, and PID, start time and command line must all match before a process is treated as the registered one.

3.8 Dashboard

The console has eight pages: Overview, Process Monitor, Applications, Incidents, Recovery Policies, Analytics, Alerts and Settings. It runs in LIVE MODE when the API is reachable and otherwise in a browser-only DEMO MODE; simulated values are never shown as live.

4. Testing and results

All 3 backend pytest tests passed, and an abrupt kill was recovered end to end on a live server. These are local functional checks, not performance benchmarks.

4.1 Test scenarios

Scenario	How it is triggered	Observed behaviour
Abrupt kill	`POST /api/demo/live-recovery`	Detected on the next poll, restarted, incident Recovered
Manual restart	`POST /api/applications/{id}/restart`	Incident Recovered (pytest)
Failing health check	Unreachable `health_check_url`	Attempt 1 Failed; no automatic retry yet (known gap)
Unauthorized script	Register script `rm -rf /`	Rejected with HTTP 422 (pytest)

Scenario	How it is triggered	Observed behaviour
Planned stop	`expected_running = false`	By code, no incident is raised (not tested)
Backend health	`GET /api/health`	Status ok, monitoring agent running (pytest)

Tools used: pytest and curl against a running server. Not yet tested: crash loops and simultaneous failures.

4.2 Sample incident timeline (re-test, 16 Oct, UTC)

Time	Level	Event
18:21:42	WARN	Failure detected: process has stopped
18:21:42	INFO	Restart attempt initiated: 1 of 3
18:21:42	OK	Recovery verified, health check passed
18:21:42	ALERT	Recovery succeeded (Low), after attempt 1

4.3 Challenges and solutions

Challenge	Solution
PID reuse	Match PID, start time and command line
Zombie processes	A zombie counts as not alive
Restart storms	Attempt limit and cooldown (automatic retry is the next step)
Intentional stops	`expected_running` flag for planned stops
Permissions and safety	Allowlisted script only; no commands from the frontend
Monitoring overhead	Light polling with a configurable interval

5. Conclusion, limitations and future work

5.1 Conclusion

Nexus detects the termination of an allowlisted process, restarts it automatically, verifies the result and keeps a timeline and alert for every incident. It is safe by design (allowlist, identity checks, attempt limits), observable (incident timelines and alerts in a dashboard) and lightweight (Python, FastAPI, SQLite and psutil, with no heavy dependencies).

5.2 Limitations

- Only the disposable `test_process.py` worker can be restarted.
- Automatic retry up to the policy limit is not implemented; a failed first attempt leaves the incident open.
- Dashboard edit forms for applications and policies are not yet synchronised with the backend write endpoints, and the dashboard summary endpoint needs a fix.
- No authentication or roles; single host; crash loops and simultaneous failures are untested.

5.3 Future work

Stage	Items
Now (working)	psutil polling, verified auto-restart, incidents with timeline, alerts, React dashboard in LIVE mode
Next (hardening)	Auto-retry up to the policy limit, authentication and roles, fix summary endpoint, sync dashboard forms
Later (scale)	PostgreSQL, containers, Teams alerts, Azure deployment, real services, multi-host monitoring

5.4 References

1. Linux `signal(7)` and `proc(5)` manual pages.
2. psutil documentation, https://psutil.readthedocs.io
3. FastAPI documentation, https://fastapi.tiangolo.com
4. SQLAlchemy documentation, https://docs.sqlalchemy.org

5. systemd.service(5), supervisord, Monit, Docker restart policies and Prometheus documentation (the tools compared in Section 2.2).